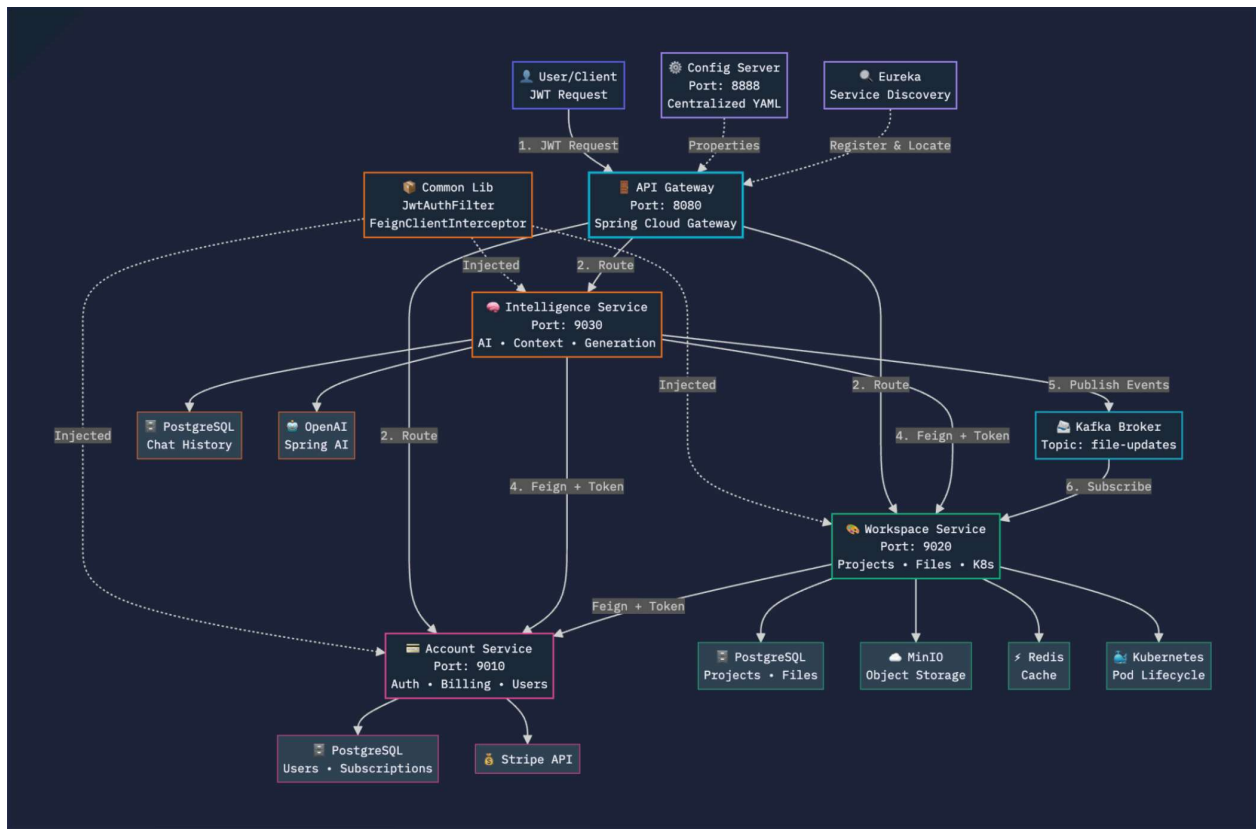


Distributed Lovable Architecture

Microservices

Microservice	Primary Responsibility	Database / Storage	Key Integrations	Cross-Service Calls (Feign)
Account Service	User Identity, Authentication, and Stripe Billing.	PostgreSQL (Users, Plans, Subscriptions)	Stripe API	<i>None</i>
Workspace Service	Project management, File Tree, Kubernetes Deployments.	PostgreSQL (Projects, Members), MinIO (Files)	Kubernetes API, Redis	Calls AccountService (Check limits, users)
Intelligence Service	AI Chat, Context gathering, Code Generation.	PostgreSQL (Chat Sessions, Messages, Events)	OpenAI (Spring AI)	Calls WorkspaceService (Read files), AccountService (Check AI limits)



Account Service

Category	Components & Details
Dependencies	<code>spring-boot-starter-web</code> , <code>spring-data-jpa</code> , <code>postgresql</code> , <code>stripe-java</code> , <code>mapstruct</code> , <code>spring-cloud-starter-netflix-eureka-client</code>
Key Entities	<code>User</code> , <code>Plan</code> (Free, Pro), <code>Subscription</code>
Security Setup	<code>AccountSecurityConfig</code> : Exposes <code>/auth/**</code> and <code>/webhooks/stripe</code> publicly. Secures everything else.
Internal API	<code>InternalAccountController</code> : Exposes <code>/internal/v1/...</code> for other microservices to fetch user DTOs and verify billing plans.

Workspace Service

Category	Components & Details
Dependencies	<code>spring-boot-starter-web</code> , <code>spring-data-jpa</code> , <code>minio</code> , <code>kubernetes-client</code> (Fabric8), <code>spring-data-redis</code> , <code>spring-cloud-starter-openfeign</code>
Key Entities	<code>Project</code> , <code>ProjectMember</code> , <code>ProjectFile</code> , <code>Preview</code> (K8s pod mapping)

Cloud Configs	<code>StorageConfig</code> (MinIO connection), <code>KubernetesConfig</code> , <code>RedisConfig</code>
Security Logic	<code>SecurityExpressions</code> : Contains custom <code>@PreAuthorize("@security.canEditProject(#id)")</code> logic to verify ownership via <code>ProjectMemberRepository</code> .
Feign Clients	<code>AccountClient</code> : Used during Project Creation to verify if the user has hit their <code>maxProjects</code> limit.

Intelligence Service

Category	Components & Details
Dependencies	<code>spring-ai-starter-model-openai</code> , <code>spring-data-jpa</code> , <code>spring-cloud-starter-openfeign</code>
Key Entities	<code>ChatSession</code> , <code>ChatMessage</code> , <code>ChatEvent</code> , <code>UsageLog</code> (Tracks LLM thoughts, file edits, tool uses)
AI Components	<code>AiGenerationServiceImpl</code> (Prompt execution), <code>LLMResponseParser</code> (Regex to extract <code><file></code> and <code><tool></code> tags).
Advisors & Tools	<code>FileTreeContextAdvisor</code> (Injects current file tree into prompt), <code>CodeGenerationHelperTools</code> (Allows LLM to read specific files).
Feign Clients	<code>WorkspaceClient</code> (To fetch file trees and file content for context), <code>AccountClient</code> (To verify AI daily token limits).

Common Lib

Component in common-lib	What it does	Why it's needed
<code>JwtAuthFilter</code>	Extracts the Bearer token and populates the <code>SecurityContext</code> .	Every service authenticates requests locally, completely independent of the Gateway.
<code>FeignClientInterceptor</code>	Automatically grabs the user's JWT from the context and attaches it to outbound Feign requests.	Allows the Intelligence service to call the Workspace service seamlessly as the logged-in user.
Shared DTOs	<code>UserDto</code> , <code>PlanDto</code> , <code>FileTreeDto</code> , etc.	Prevents JPA <code>@Entity</code> classes from leaking across microservice boundaries.